CASO DE ESTUDIO

¿Se puede predecir una falla industrial antes de que ocurra?

En este proyecto apliqué analítica de datos y Machine Learning a un dataset de sensores industriales para detectar fallas y diagnosticar su causa, comparando el rendimiento de distintos modelos de clasificación.

EL PROBLEMA

En la industria manufacturera, una máquina que falla sin aviso puede costar **entre $10,000 y $250,000 USD por cada hora de inactividad**: líneas detenidas, pedidos incumplidos, repuestos de emergencia, turnos extendidos.

A pesar de este impacto, muchas plantas aún operan bajo esquemas de mantenimiento reactivo (reparo cuando se rompe) o preventivo por calendario (cambio piezas cada X meses sin importar su estado). Ambos enfoques presentan limitaciones en términos de eficiencia y costo.

> *¿Y si los datos que la máquina ya genera pudieran anticipar la falla antes de que ocurra — e incluso predecir qué tipo de falla será?*

Para responder esta pregunta, trabajé con el dataset AI4I 2020 disponible en Kaggle, un dataset sintético diseñado para replicar condiciones operativas industriales, utilizando dos enfoques complementarios.

DOS PREGUNTAS, DOS MODELOS

¿VA A FALLAR? Detección de fallas	¿QUÉ TIPO DE FALLA? Diagnóstico específico
¿Se debe intervenir la máquina antes del siguiente turno?	¿Qué repuestos y recursos técnicos serán necesarios para la intervención?
Clasifica entre falla y no falla	Clasifica entre diversos tipos de falla
Solo 3.4% de registros corresponden a fallas reales	Un tipo de falla (falla aleatoria) tiene apenas 18 registros.

LO QUE DESCUBRÍ EN LOS DATOS

Relaciones determinísticas y riesgo de data leakage

El primer modelo que entrené daba resultados casi perfectos. Demasiado perfectos. Al hacer el análisis, descubrí que 5 columnas del dataset coincidían un 99.7% con la variable que quería predecir.

Esto se llama **data leakage**: el modelo estaba "haciendo trampa", usando información que en la realidad solo estaría disponible *después* de que la falla ya ocurrió.

> *Lección: métricas elevadas deben interpretarse con cautela si no se valida la disponibilidad real de las variables en producción.*

El conocimiento del proceso importó más que el algoritmo

Después de eliminar el data leakage, creé nuevas variables combinando las señales de los sensores con lógica de ingeniería:

Variable creada	Qué mide	Para qué sirve
`torque_speed_ratio`	Estrés mecánico (torque vs. velocidad).	Detectar sobreesfuerzo.
`power`	Potencia (Torque × RPM).	Identificar fallas de potencia.
`overheat_indicator`	Temperatura alta + velocidad baja.	Anticipar sobrecalentamiento.
`wear_torque`	Desgaste acumulado × torque.	Medir estrés combinado.

Estas variables creadas manualmente **ocuparon las primeras 5 posiciones en importancia del modelo**. El entendimiento del proceso físico es un factor diferencial en problemas de mantenimiento predictivo.

RESULTADOS — PARTE 1: ¿LA MÁQUINA VA A FALLAR?

El primer modelo responde una pregunta sencilla pero crítica: dado el estado actual de los sensores, ¿hay riesgo de falla? Es una clasificación binaria donde el modelo debe diferenciar entre máquinas que están operando normalmente y máquinas a punto de fallar.

Entrené 5 modelos distintos para comparar: Logistic Regression, Decision Tree, Random Forest, Gradient Boosting y CatBoost. Todos con la misma preparación de datos y el mismo tratamiento de desbalance de clases (class_weight balanceado).

Modelo	AUC-ROC	F1	Recall	Precision	CV AUC
Random Forest	**0.987**	**0.882**	**0.824**	**0.949**	**0.983**
CatBoost	0.973	0.861	0.868	0.855	0.976
Gradient Boosting	0.962	0.880	0.809	0.965	0.975
Logistic Regression	0.953	0.322	0.882	0.197	0.942
Decision Tree	0.907	0.582	0.809	0.455	0.928

¿Qué nos dicen estos números?

Los tres modelos basados en árboles de decisión (Random Forest, CatBoost y Gradient Boosting) dominan a los modelos lineales. **Random Forest ganó en AUC-ROC general (0.987)**, pero las diferencias con CatBoost y Gradient Boosting son mínimas. Logistic Regression y Decision Tree quedaron muy por debajo en precisión.

En contexto del problema, lo más relevante son los errores. Random Forest detecta **56 de 68 fallas reales** en el conjunto de prueba y solo genera **3 falsas alarmas de 59 alertas totales**. Es un modelo muy conservador: cuando dice “hay falla”, acierta 19 de cada 20 veces.

Random Forest vs. CatBoost: el trade-off clave

La decisión interesante no está en elegir entre el mejor y el peor modelo — sino entre los dos mejores. Random Forest y CatBoost tienen perfiles distintos que se ajustan a escenarios de negocio diferentes:

Random Forest lidera en AUC-ROC (0.987) y Precision (94.9%): cuando dice que hay falla, **acierta 19 de cada 20 veces**. Solo 3 falsas alarmas de 59 alertas.

CatBoost tiene mejor Recall (86.8% vs 82.4%): **detecta más fallas reales** — 59 de 68 vs 56 de 68. Pero a costa de más falsas alarmas (Precision 85.5%).

RANDOM FOREST	CATBOOST
AUC 0.987 · Precision 94.9%	**Recall 86.8% · F1 0.861**
Detecta 56/68 fallas reales	Detecta 59/68 fallas reales
Solo 3 falsas alarmas de 59 alertas. Ideal cuando las intervenciones innecesarias tienen un costo alto.	3 fallas reales más detectadas que RF, pero con más falsas alarmas. Ideal cuando una falla no detectada es mucho más costosa.

> *¿Cuál es mejor? Depende del costo real de cada error en la planta. Si una hora de downtime cuesta $50K y una inspección preventiva $500, CatBoost es preferible: las 3 fallas adicionales que detecta valen mucho más que las falsas alarmas extra que genera.*

RESULTADOS — PARTE 2: ¿QUÉ TIPO DE FALLA SERÁ?

Detectar que una máquina va a fallar es útil, pero incompleto. El equipo de Mantenimiento necesita saber QUÉ tipo de falla esperar para despachar el técnico correcto y preparar el repuesto adecuado. Este segundo modelo aborda exactamente esa pregunta: dado que hay riesgo de falla, ¿de qué tipo será?

Es un problema de clasificación multiclase con 7 categorías: 5 tipos individuales de falla, una categoría para fallas múltiples simultáneas, y la clase “sin falla”. La complejidad aquí no está en el algoritmo, sino en el desbalance extremo: hay clases con menos de 20 ejemplos en todo el dataset.

Modelo	AUC OvR	F1-macro	Recall macro	Precision macro
Random Forest	**0.914**	**0.681**	**0.710**	**0.670**
Gradient Boosting	0.911	0.669	0.677	0.663
Decision Tree	0.858	0.665	0.732	0.672
CatBoost	0.926	0.615	0.717	0.564
Logistic Regression	0.939	0.493	0.827	0.455

¿Qué nos dicen estos números?

El ranking se invierte respecto al caso binario. Aquí **Random Forest gana en F1-macro (0.681)** seguido de cerca por Gradient Boosting y Decision Tree. CatBoost — que en el caso binario era competitivo — cae al cuarto lugar en F1-macro pero sorprende con **el mejor AUC One-vs-Rest del grupo (0.926, mejor que RF)**. Esto significa que CatBoost ordena bien las probabilidades pero es más conservador en la decisión final.

Logistic Regression es un caso interesante: tiene el AUC más alto y el mejor Recall (0.83), pero el F1 más bajo. Esto significa que **predice muchas cosas como falla**, lo que sube el recall pero genera tantos falsos positivos que la precision se desploma.

F1-Score por tipo de falla y modelo

La métrica F1-macro promedia el rendimiento de todas las clases por igual. Pero al verlo desglosado por tipo de falla aparece algo más interesante: hay tres clases que todos los modelos clasifican excelente, y dos que ninguno logra resolver.

Modelo	HDF	PWF	OSF	No Fail	Multiple	TWF	RNF
Random Forest	**0.98**	**1.00**	**0.91**	**0.99**	0.80	0.09	0.00
Gradient Boosting	**0.98**	**0.97**	**0.94**	**1.00**	0.80	0.00	0.00
Decision Tree	**0.95**	**1.00**	**1.00**	**0.94**	0.67	0.09	0.00
CatBoost	**0.98**	0.74	0.71	**0.98**	0.80	0.09	0.00
Logistic Regression	**1.00**	0.56	0.58	0.75	0.48	0.08	0.01

Tres fallas “fáciles de identificar” y dos imposibles

Tres tipos de falla se clasifican con rendimiento excelente en **casi todos los modelos: HDF, PWF y OSF**. Estas tres clases tienen firmas claras y distinguibles en los sensores, y los indicadores de ingeniería que creé (power_anomaly, overheat_indicator, overstrain_indicator) las capturan casi perfectamente.

En contraste, **TWF (desgaste de herramienta) solo logra F1 = 0.09**. El problema no es el algoritmo: TWF comparte su señal principal (desgaste alto) con OSF, y con solo 46 ejemplos en todo el dataset, el modelo no logra separarlas.

Y **RNF (fallas aleatorias) llega a F1 = 0.00**. Esto no es un error del modelo: las fallas aleatorias *no tienen patrón por definición*. Con 18 ejemplos sin correlación alguna con los sensores, ningún modelo podría predecirlas.

> *Reconocer los límites del modelo es parte del trabajo. Las fallas aleatorias no se resuelven con más datos o mejores algoritmos — se resuelven con redundancia operativa: repuestos en stock y mantenimiento preventivo basado en vida útil (MTBF).*

Cada tipo de falla dejó una “huella digital” en los sensores

Al analizar los datos por categoría, descubrí que cada tipo de falla tiene un patrón característico en las lecturas de los sensores. Esto explica por qué algunas clases se predicen casi perfectamente y otras son imposibles de clasificar.

Tipo de falla	“Huella digital” en sensores	F1-Score
PWF (potencia)	Potencia fuera de rango (extrema alta o baja)	**1.00**
HDF (calor)	Temperatura alta + velocidad rotacional MUY baja	**0.98**
OSF (sobreesfuerzo)	Torque alto + desgaste acumulado alto	**0.94**
TWF (desgaste)	Desgaste extremo, pero se confunde con OSF	**0.09**
RNF (aleatoria)	Sin patrón detectado en ningún sensor	**0.00**

Juntando ambos modelos

El modelo binario y el multiclase no compiten — se complementan. El binario actúa como filtro de primera línea con alta sensibilidad (detecta si hay riesgo), y el multiclase diagnostica el tipo específico solo cuando el binario lo activa.

Esta arquitectura jerárquica tiene una ventaja práctica: si el binario (AUC 0.99) no detecta riesgo, no necesitamos correr el multiclase. Y cuando el multiclase sí se ejecuta, trabaja con casos ya prefiltrados, lo que mejora su rendimiento efectivo en producción.

¿CÓMO FUNCIONARÍA EN UNA PLANTA REAL?

PASO 1	PASO 2	PASO 3
Detectar	**Diagnosticar**	**Actuar**
El modelo binario (RF o CatBoost) analiza la data de sensores: ¿hay riesgo de falla?	Si detecta riesgo, el multiclase clasifica: ¿calor, potencia o sobreesfuerzo?	El equipo recibe la alerta con el tipo de falla y el recurso que necesita.

> *La diferencia entre "la máquina falló, hay que repararla" y "va a fallar por sobrecalentamiento en las próximas horas y ya tenemos el técnico de refrigeración listo" es la diferencia entre mantenimiento reactivo y mantenimiento inteligente.*

APRENDIZAJES CLAVE

1. **Verificar si los datos tienen trampas ocultas**
   El data leakage es el error más peligroso porque es silencioso: métricas excelentes que no funcionan en producción. Detectarlo requiere entender qué datos estarían disponibles en el momento real de la predicción.

2. **Entender el proceso físico vale más que probar más algoritmos**
   Las variables de ingeniería (torque_speed_ratio, overheat_indicator) fueron más predictivas que las señales crudas del sensor. Un data scientist con conocimiento de dominio tiene ventaja competitiva real.

3. **No existe "el mejor modelo" sin contexto de negocio**
   Random Forest tiene mejor AUC y Precision. CatBoost tiene mejor Recall. La elección correcta depende del costo relativo de cada tipo de error en la planta específica, no de una métrica aislada.

4. **No todo es predecible — y reconocerlo es una decisión profesional**
   Las fallas aleatorias no tienen patrón. La respuesta correcta es redundancia operativa: repuestos en stock y mantenimiento por tiempo (MTBF). La predicción complementa, no reemplaza.

5. **El valor está en el sistema completo, no en un modelo aislado**
   Modelo binario que detecta + multiclase que diagnostica + regla de negocio que despacha el recurso. La analítica no reemplaza al ingeniero: le da mejor información para decidir.

Proyecto completo (código, notebooks, datos): github.com/c-tong02/Portafolio-de-Analitica

Dataset: **AI4I 2020 Predictive Maintenance (Kaggle)** · Metodología: **CRISP-DM** · Herramientas: **Python, scikit-learn, CatBoost**

#MantenimientoPredictivo #MachineLearning #DataScience #Industria40 #CRISPDM